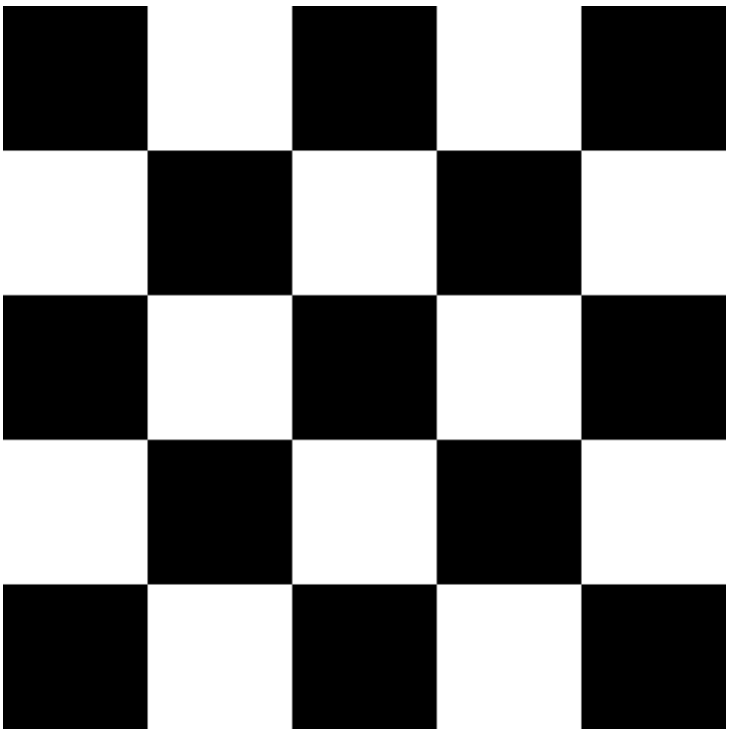
In [2]:
```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Image
%matplotlib inline
```

In [5]:
```
Image(filename="checkerboard_18x18.png")
```

Out[5]:



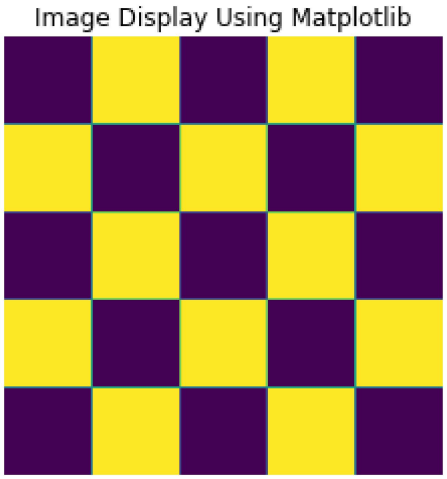
In [6]:
```
# Reading image as gray scale.
cb_img = cv2.imread("checkerboard_18x18.png", cv2.IMREAD_GRAYSCALE)

# Print the image data (pixel values), element of a 2D numpy array.
# Each pixel value is 8-bits [0,255]
print(cb_img)
cv2.imshow('Grayscale Image', cb_img)
```

```
[[0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 ...
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]]
```

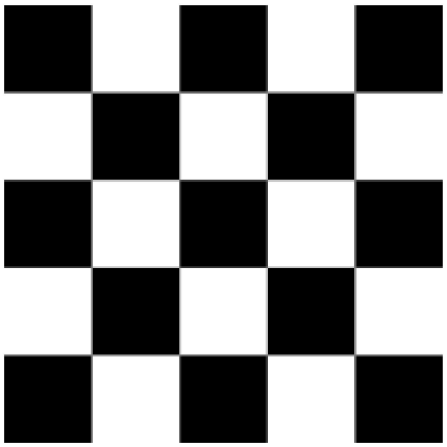
In [7]:
```
# Displaying the image attributes
print("Shape of the image: ", cb_img.shape)
print("Data type of the image: ", cb_img.dtype)
print("Size of the image: ", cb_img.size)
```

```
Shape of the image:  (360, 360)
Data type of the image:  uint8
Size of the image:  129600
```

In [8]:
```
# Displaying image using Matplotlib.
plt.imshow(cb_img)
plt.title("Image Display Using Matplotlib")
plt.axis("off") # To turn off axes
plt.show()
```

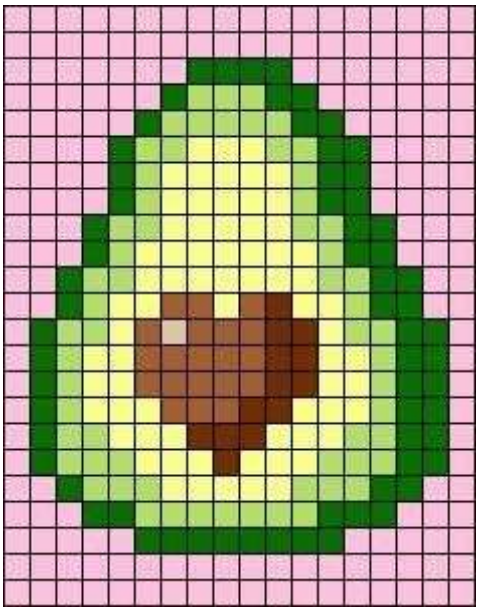


In [9]:
```
# Set color map to gray scale for proper rendering.
plt.imshow(cb_img, cmap = "gray")
plt.axis("off")
plt.show()
```



In [10]:
```
# Read and display image
Image("avacado.jpg")
```

Out[10]:

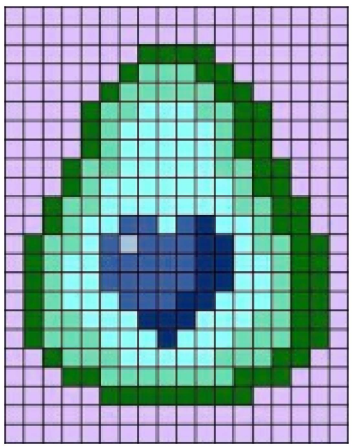


In [11]:
```
# Reading the image
avacado_img = cv2.imread("avacado.jpg", cv2.IMREAD_COLOR)

# Printing the shape of the image
print("Image shape (height, width, channels) is:", avacado_img.shape)
```

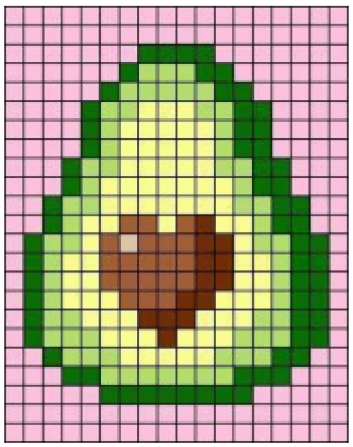
```
# Printing the size of the image
print("Image size is: ", avacado_img.size)

# Printing the data-type of the image
print("Data type of image is:", avacado_img.dtype)
```

```
Image shape (height, width, channels) is: (300, 235, 3)
Image size is:  211500
Data type of image is: uint8
```

In [12]:
```
# Displaying the image using matplotlib
plt.imshow(avacado_img)
plt.axis("off")
plt.show()
```

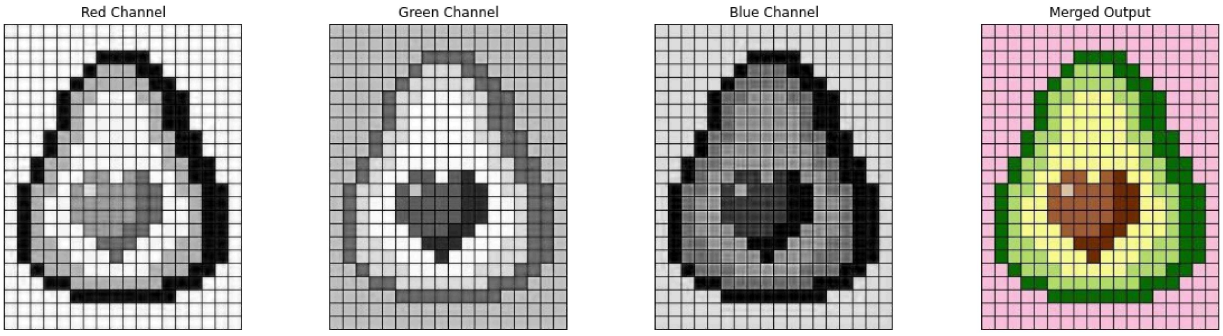


In [13]:
```
# Reversing the channels of the color image
avacado_img_channels_reversed = avacado_img[:, :, ::-1]
plt.imshow(avacado_img_channels_reversed)
plt.axis("off")
plt.show()
```

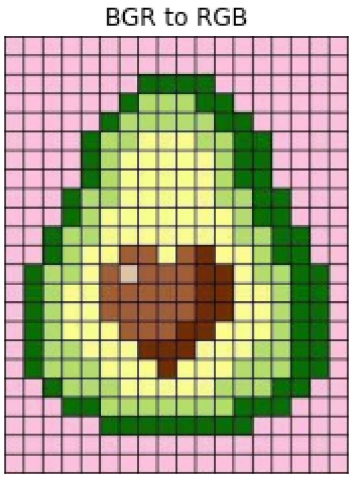


In [14]:
```
# Split the image into the B,G,R components
b, g, r = cv2.split(avacado_img)

# Show the channels
plt.figure(figsize = [20, 5])

# Red Channel
plt.subplot(1, 4, 1)
plt.imshow(r, cmap = "gray")
plt.title("Red Channel")
plt.axis("off")
```

```
# Green Channel
plt.subplot(1, 4, 2)
plt.imshow(g, cmap = "gray")
plt.title("Green Channel")
plt.axis("off")

# Blue Channel
plt.subplot(1, 4, 3)
plt.imshow(b, cmap = "gray")
plt.title("Blue Channel")
plt.axis("off")

# Merge the individual channels into a BGR image
merged_img = cv2.merge((b, g, r))

# Show the merged output
plt.subplot(1, 4, 4)
plt.imshow(merged_img[:, :, ::-1])
plt.title("Merged Output")
plt.axis("off")
plt.show()
```

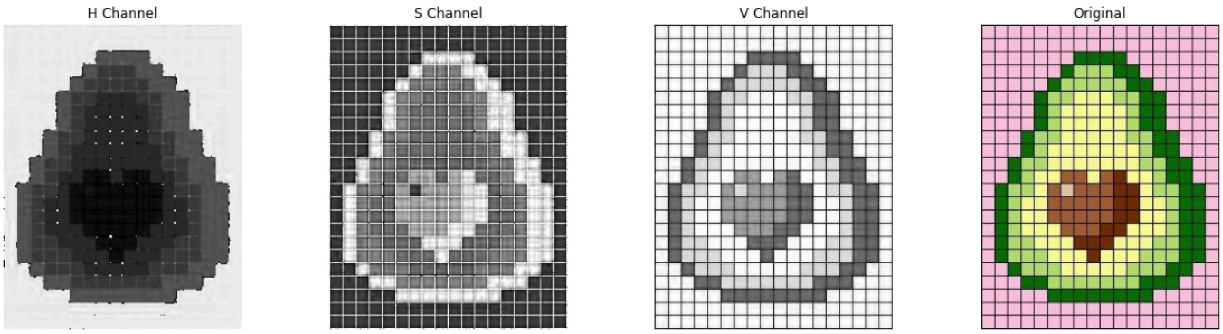


In [15]:
```
# OpenCV stores color channels in a differnet order than most other applications (BG
avacado_img_bgr = cv2.imread("avacado.jpg", cv2.IMREAD_COLOR)

# Converting BGR to RGB
avacado_img_rgb = cv2.cvtColor(avacado_img_bgr, cv2.COLOR_BGR2RGB)
plt.imshow(avacado_img_rgb)
plt.title("BGR to RGB")
plt.axis("off")
plt.show()
```



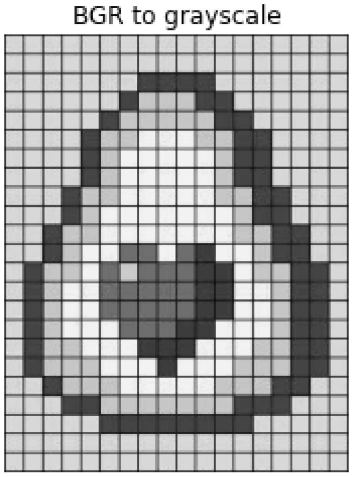
In [16]:
```
# Converting to HSV
avacado_img_hsv = cv2.cvtColor(avacado_img_bgr, cv2.COLOR_BGR2HSV)

# Split the image into H, S, V components
h, s, v = cv2.split(avacado_img_hsv)
```

```
# Show the channels
plt.figure(figsize = [20, 5])

# Hue Channel
plt.subplot(1, 4, 1)
plt.imshow(h, cmap = "gray")
plt.title("H Channel")
plt.axis("off")

# Saturation Channel
plt.subplot(1, 4, 2)
plt.imshow(s, cmap = "gray")
plt.title("S Channel")
plt.axis("off")

# Value Channel
plt.subplot(1, 4, 3)
plt.imshow(v, cmap = "gray")
plt.title("V Channel")
plt.axis("off")

# Show the original image
plt.subplot(1, 4, 4)
plt.imshow(avacado_img_rgb)
plt.title("Original")
plt.axis("off")
plt.show()
```

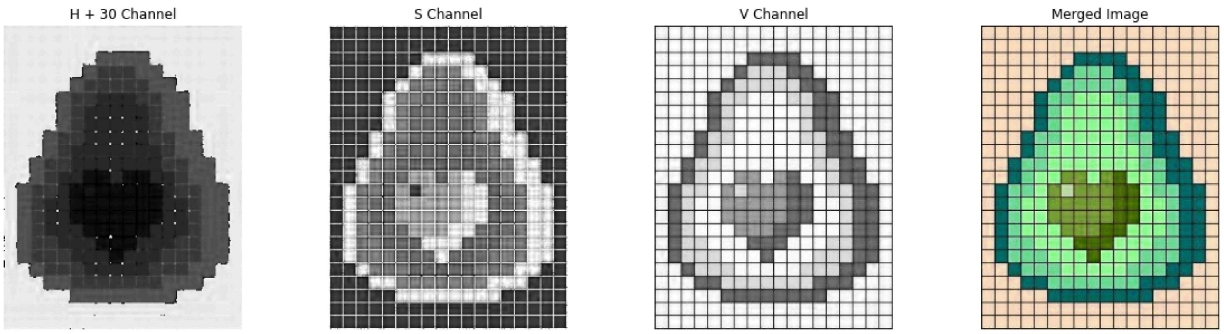


In [18]:

```
# Converting BGR to grayscale
avacado_img_gray = cv2.cvtColor(avacado_img_bgr, cv2.COLOR_BGR2GRAY)
plt.imshow(avacado_img_gray, cmap = "gray")
plt.title("BGR to grayscale")
plt.axis("off")
plt.show()
```



In [19]:
```
h_new = h + 30
avacado_img_merged = cv2.merge((h_new, s, v))
avacado_img_rgb = cv2.cvtColor(avacado_img_merged, cv2.COLOR_HSV2RGB)

# Split the image into the B,G,R components
h,s,v = cv2.split(avacado_img_merged)

# Show the channels
plt.figure(figsize = [20, 5])

# Hue Channel
plt.subplot(1, 4, 1)
plt.imshow(h, cmap = "gray")
plt.title("H + 30 Channel")
plt.axis("off")

# Saturation Channel
plt.subplot(1, 4, 2)
plt.imshow(s, cmap = "gray")
plt.title("S Channel")
plt.axis("off")

# Value Channel
plt.subplot(1, 4, 3)
plt.imshow(v, cmap = "gray")
plt.title("V Channel")
plt.axis("off")

# Show the original image
plt.subplot(1, 4, 4)
plt.imshow(avacado_img_rgb)
plt.title("Merged Image")
plt.axis("off")
plt.show()
```



In [20]:
```
# Saving the image
cv2.imwrite("avacado_saved.jpg", avacado_img_rgb)
```

Out[20]: True

In [ ]:

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("messi5.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# --------------------
# 1. Edge Detection
# --------------------
edges = cv2.Canny(gray, 100, 200)

# --------------------
# 2. Line Detection
# --------------------
lines = cv2.HoughLines(edges, 1, np.pi/180, 150)
img_lines = img.copy()

if lines is not None:
    for rho, theta in lines[:,0]:
        a, b = np.cos(theta), np.sin(theta)
        x0, y0 = a*rho, b*rho
        x1 = int(x0 + 1000*(-b))
        y1 = int(y0 + 1000*(a))
        x2 = int(x0 - 1000*(-b))
        y2 = int(y0 - 1000*(a))
        cv2.line(img_lines, (x1,y1), (x2,y2), (0,0,255), 2)

# --------------------
# 3. Corner Detection
# --------------------
corners = cv2.cornerHarris(gray, 2, 3, 0.04)
img_corners = img.copy()
img_corners[corners > 0.01 * corners.max()] = [0,255,0]

# --------------------
# 4. Scaling
# --------------------
scaled = cv2.resize(img, None, fx=0.5, fy=0.5)

# --------------------
# 5. Translation
# --------------------
rows, cols = gray.shape
M = np.float32([[1,0,50],[0,1,30]])
translated = cv2.warpAffine(img, M, (cols, rows))

# --------------------
# 6. Rotation
```

```
# --------------------
M_rot = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1)
rotated = cv2.warpAffine(img, M_rot, (cols, rows))

# --------------------
# 7. Perspective Transform
# --------------------
pts1 = np.float32([[50,50],[200,50],[50,200],[200,200]])
pts2 = np.float32([[10,100],[200,50],[100,250],[200,200]])

M_p = cv2.getPerspectiveTransform(pts1, pts2)
perspective = cv2.warpPerspective(img, M_p, (cols, rows))

# --------------------
# Display
# --------------------
titles =
["Original","Edges","Lines","Corners","Scaled","Translated","Rotated",
"Perspective"]
images = [img, edges, img_lines, img_corners, scaled, translated,
rotated, perspective]

for i in range(8):
    plt.subplot(2,4,i+1)
    if i == 1:
        plt.imshow(images[i], cmap='gray')
    else:
        plt.imshow(cv2.cvtColor(images[i], cv2.COLOR_BGR2RGB))
    plt.title(titles[i])
    plt.axis("off")

plt.show()
```

Original

Edges

Lines

Corners

Scaled

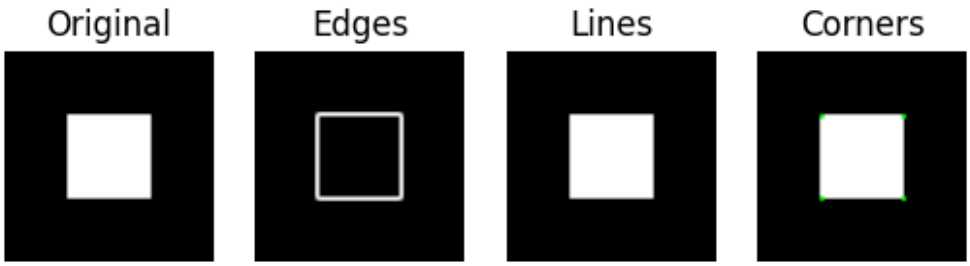
Translated

Rotated

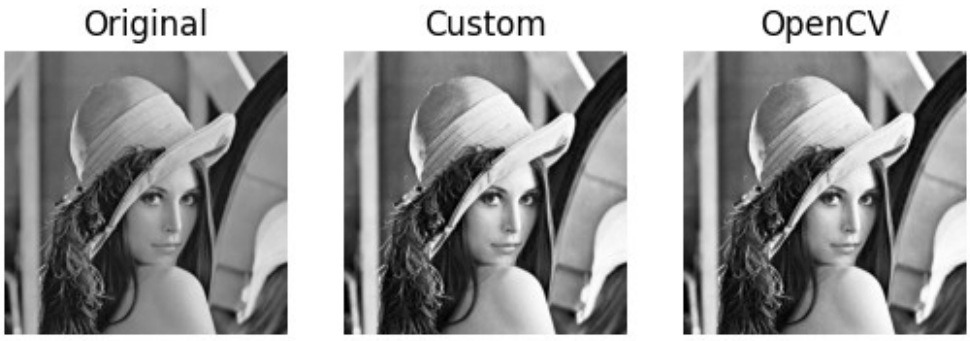
Perspective

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create simple image (white square)
img = np.zeros((100,100), dtype=np.uint8)
img[30:70,30:70] = 255

# --------------------
# 1. Edge Detection (Sobel)
# --------------------
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1)
edges = cv2.magnitude(sobelx, sobely)

# --------------------
# 2. Line Detection (Hough)
# --------------------
edges_uint8 = np.uint8(edges)
lines = cv2.HoughLines(edges_uint8, 1, np.pi/180, 100)

img_lines = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)

if lines is not None:
    for rho, theta in lines[:,0]:
        a, b = np.cos(theta), np.sin(theta)
        x0, y0 = a*rho, b*rho
        x1 = int(x0 + 100*(-b))
        y1 = int(y0 + 100*(a))
        x2 = int(x0 - 100*(-b))
        y2 = int(y0 - 100*(a))
        cv2.line(img_lines, (x1,y1), (x2,y2), (0,0,255), 1)

# --------------------
# 3. Corner Detection (Harris)
# --------------------
img_float = np.float32(img)
corners = cv2.cornerHarris(img_float, 2, 3, 0.04)

img_corners = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
img_corners[corners > 0.01 * corners.max()] = [0,255,0]

# --------------------
# Display Results
# --------------------
plt.subplot(1,4,1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis("off")
```

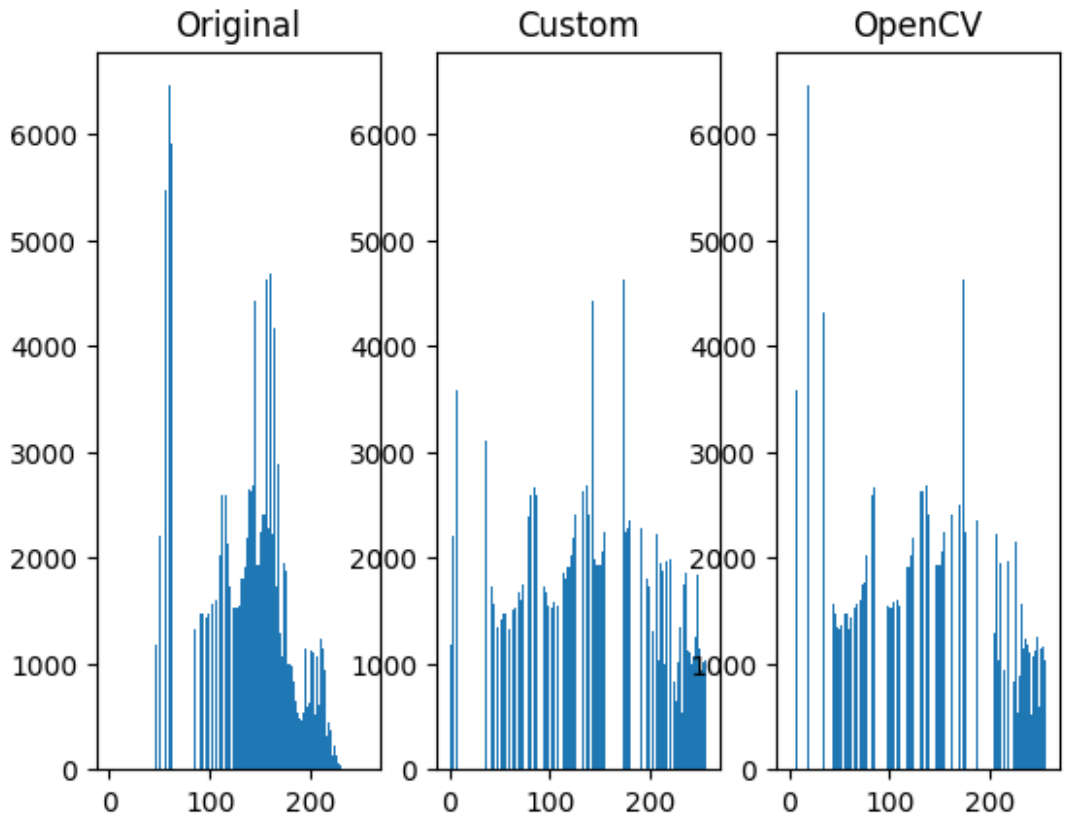
```
plt.subplot(1,4,2)
plt.imshow(edges, cmap='gray')
plt.title("Edges")
plt.axis("off")

plt.subplot(1,4,3)
plt.imshow(img_lines)
plt.title("Lines")
plt.axis("off")

plt.subplot(1,4,4)
plt.imshow(img_corners)
plt.title("Corners")
plt.axis("off")

plt.show()
```



```python
import numpy as np
import cv2
import matplotlib.pyplot as plt

# Custom Histogram Equalization
def equalize_hist(img):
    levels, counts = np.unique(img, return_counts=True)
    pdf = counts / img.size
    cdf = np.cumsum(pdf)
    transform = np.floor(255 * cdf).astype(np.uint8)
    mapping = dict(zip(levels, transform))
    return np.vectorize(mapping.get)(img).astype(np.uint8)

# Read image
img = cv2.imread("example.jpg", 0)

# Apply equalization
custom = equalize_hist(img)
opencv_eq = cv2.equalizeHist(img)

# Display images
plt.subplot(1,3,1); plt.imshow(img, cmap='gray');
plt.title("Original"); plt.axis('off')
plt.subplot(1,3,2); plt.imshow(custom, cmap='gray');
plt.title("Custom"); plt.axis('off')
plt.subplot(1,3,3); plt.imshow(opencv_eq, cmap='gray');
plt.title("OpenCV"); plt.axis('off')
plt.show()

# Display histograms
plt.subplot(1,3,1); plt.hist(img.ravel(),256,[0,256]);
plt.title("Original")
plt.subplot(1,3,2); plt.hist(custom.ravel(),256,[0,256]);
plt.title("Custom")
plt.subplot(1,3,3); plt.hist(opencv_eq.ravel(),256,[0,256]);
plt.title("OpenCV")
plt.show()
```



```
C:\Users\karth\AppData\Local\Temp\ipykernel_15876\2429380021.py:28:
MatplotlibDeprecationWarning: Passing the range parameter of hist()
positionally is deprecated since Matplotlib 3.10; the parameter will
become keyword-only in 3.12.
  plt.subplot(1,3,1); plt.hist(img.ravel(),256,[0,256]);
plt.title("Original")
C:\Users\karth\AppData\Local\Temp\ipykernel_15876\2429380021.py:29:
MatplotlibDeprecationWarning: Passing the range parameter of hist()
positionally is deprecated since Matplotlib 3.10; the parameter will
become keyword-only in 3.12.
  plt.subplot(1,3,2); plt.hist(custom.ravel(),256,[0,256]);
plt.title("Custom")
C:\Users\karth\AppData\Local\Temp\ipykernel_15876\2429380021.py:30:
MatplotlibDeprecationWarning: Passing the range parameter of hist()
positionally is deprecated since Matplotlib 3.10; the parameter will
become keyword-only in 3.12.
  plt.subplot(1,3,3); plt.hist(opencv_eq.ravel(),256,[0,256]);
plt.title("OpenCV")
```



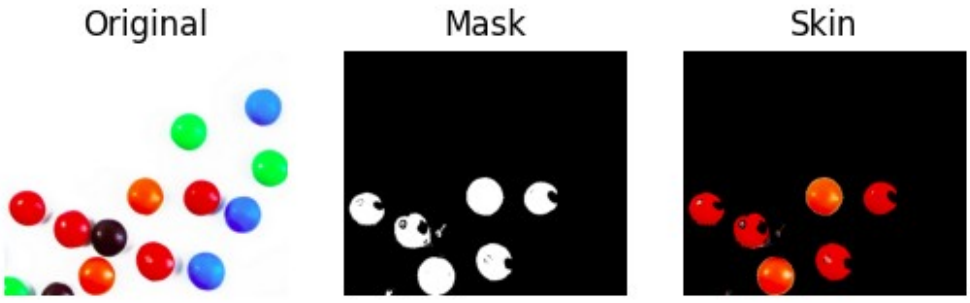
```
import os
import numpy as np
import matplotlib.pyplot as plt
from imageio import imread

face_dir = "FacePhoto"
mask_dir = "GroundT_FacePhoto"

# Load images
def load_images(img_dir, mask_dir):
    images, masks, names = [], [], []
    for file in os.listdir(img_dir):
        name = os.path.splitext(file)[0]
        mask_path = os.path.join(mask_dir, name + ".png")

        if os.path.exists(mask_path):
            img = imread(os.path.join(img_dir, file))
            mask = imread(mask_path)

            images.append(img)
            masks.append(mask)
            names.append(file)
    return images, masks, names

# Extract skin
def extract_skin(img, mask):
    skin = np.all(mask[:,:,:3] == [255,255,255], axis=-1)
    result = np.zeros_like(img)
    result[skin] = img[skin]
    return result, skin

# Load dataset
images, masks, names = load_images(face_dir, mask_dir)

# Process first 3 images
for i in range(min(3, len(images))):
    img, mask, name = images[i], masks[i], names[i]

    skin_img, skin_pixels = extract_skin(img, mask)

    percent = np.sum(skin_pixels) / skin_pixels.size * 100
    print(name, "- Skin %:", round(percent,2))

    # Display
    plt.subplot(1,3,1); plt.imshow(img); plt.title("Original");
plt.axis('off')
    plt.subplot(1,3,2); plt.imshow(mask); plt.title("Mask");
plt.axis('off')
    plt.subplot(1,3,3); plt.imshow(skin_img); plt.title("Skin");
```

```
plt.axis('off')
    plt.show()
```

```
img0.jpg - Skin %: 47.87
```

```
C:\Users\karth\AppData\Local\Temp\ipykernel_20624\3901693597.py:17:
DeprecationWarning: Starting with ImageIO v3 the behavior of this
function will switch to that of iio.v3.imread. To keep the current
behavior (and make this warning disappear) use `import imageio.v2 as
imageio` or call `imageio.v2.imread` directly.
  img = imread(os.path.join(img_dir, file))
C:\Users\karth\AppData\Local\Temp\ipykernel_20624\3901693597.py:18:
DeprecationWarning: Starting with ImageIO v3 the behavior of this
function will switch to that of iio.v3.imread. To keep the current
behavior (and make this warning disappear) use `import imageio.v2 as
imageio` or call `imageio.v2.imread` directly.
  mask = imread(mask_path)
```


```
img1.jpg - Skin %: 8.19
```




```
img2.jpg - Skin %: 57.94
```

Original

Mask

Skin

```python
import cv2
import numpy as np

# Load image
img = cv2.imread("wrapping.png")

# Check image
if img is None:
    print("Image not found")
    exit()

# Show original
cv2.imshow("Original Image", img)

# Source points (adjust based on image)
src_pts = np.float32([
    [420,170],
    [650,150],
    [680,420],
    [400,440]
])

# Destination points (A4-like)
width, height = 300, 420
dst_pts = np.float32([
    [0,0],
    [width,0],
    [width,height],
    [0,height]
])

# Perspective transform
H = cv2.getPerspectiveTransform(src_pts, dst_pts)
warped = cv2.warpPerspective(img, H, (width, height))

# Show result
cv2.imshow("Warped Image", warped)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
import cv2
import numpy as np

def main():

    video_path = "t1.mp4"   # keep video in same folder

    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Cannot open video")
        return

    ret, first_frame = cap.read()
    if not ret:
        print("Error: Cannot read video")
        return

    prev_gray = cv2.cvtColor(first_frame, cv2.COLOR_BGR2GRAY)
    prev_gray = cv2.GaussianBlur(prev_gray, (15,15), 0)

    print("Press 'q' to quit")

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        gray = cv2.GaussianBlur(gray, (15,15), 0)

        diff = cv2.absdiff(prev_gray, gray)

        thresh = cv2.threshold(diff, 15, 255, cv2.THRESH_BINARY)[1]
        thresh = cv2.dilate(thresh, None, iterations=3)

        contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

        for c in contours:
            if cv2.contourArea(c) > 200:
                x,y,w,h = cv2.boundingRect(c)
                cv2.rectangle(frame, (x,y), (x+w,y+h), (0,255,0), 2)

        cv2.imshow("Motion", frame)

        prev_gray = gray

        if cv2.waitKey(30) & 0xFF == ord('q'):
            break
```

```
    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

```
Press 'q' to quit
```

```python
# ------------------------------------------------------------
# AIM:
# To perform object detection using YOLOv8 and display output
# directly using OpenCV
# ------------------------------------------------------------

# ------------------------------------------------------------
# Import Libraries
# ------------------------------------------------------------
import cv2
import random
from ultralytics import YOLO

# ------------------------------------------------------------
# Load YOLO Model
# ------------------------------------------------------------
model = YOLO("yolov8n.pt")

# ------------------------------------------------------------
# 1. OBJECT DETECTION ON DEFAULT IMAGE
# ------------------------------------------------------------
print("\nRunning detection on default image...")

# YOLO will automatically download this image
results = model("https://ultralytics.com/images/bus.jpg")

# Get output image
output_img = results[0].plot()

# Convert RGB → BGR for OpenCV
output_img = cv2.cvtColor(output_img, cv2.COLOR_RGB2BGR)

# Show output
cv2.imshow("YOLO Detection - Image", output_img)
cv2.waitKey(0)
cv2.destroyAllWindows()

# ------------------------------------------------------------
# 2. OBJECT DETECTION ON VIDEO
# ------------------------------------------------------------
print("\nRunning detection on video...")

video_path = "input.mp4"   # Place your video in same folder

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Video not found")
    exit()
```

```
fps = int(cap.get(cv2.CAP_PROP_FPS))
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

out = cv2.VideoWriter("output.mp4",
                      cv2.VideoWriter_fourcc(*'mp4v'),
                      fps, (w, h))

# Color generator
def get_color(cls_id):
    random.seed(cls_id)
    return tuple(random.randint(0, 255) for _ in range(3))

frame_count = 0
MAX_FRAMES = 100

while True:
    ret, frame = cap.read()
    if not ret or frame_count >= MAX_FRAMES:
        break

    results = model.track(frame, stream=True, verbose=False)

    for result in results:
        for box in result.boxes:

            if box.conf[0] > 0.4:
                x1, y1, x2, y2 = map(int, box.xyxy[0])
                cls = int(box.cls[0])
                conf = float(box.conf[0])

                color = get_color(cls)

                cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

                label = f"{result.names[cls]} {conf:.2f}"
                cv2.putText(frame, label, (x1, y1 - 10),
                            cv2.FONT_HERSHEY_SIMPLEX,
                            0.6, color, 2)

    # Show live output
    cv2.imshow("YOLO Detection - Video", frame)

    out.write(frame)
    frame_count += 1

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
out.release()
```

```
cv2.destroyAllWindows()

print("\n Output video saved as output.mp4")
```

```
Running detection on default image...

Found https://ultralytics.com/images/bus.jpg locally at bus.jpg
image 1/1 C:\Users\karth\bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 45.9ms
Speed: 1.5ms preprocess, 45.9ms inference, 0.7ms postprocess per image at shape (1, 3, 640, 480)

Running detection on video...
Error: Video not found

 Output video saved as output.mp4
```